ACM 区域赛个人代码模板

基于本仓库练习、训练赛、XCPC 代码与个人算法笔记整理

用途

比赛中遗忘写法时快速定位：先按目录找到题型，再复制对应的个人代码片段。代码来自仓库中的笔记或源码；说明文字用于提醒适用条件、复杂度和容易写错的边界。

整理规则

同主题存在多个版本时，优先参考仓库中修改时间较晚、且已整理成笔记或较新训练/比赛文件中的写法。文件时间受同步与复制影响，作为权重而非正确性证明；冲突文件不纳入模板。题目专属解法只作为思路索引，不能直接当通用模板。

覆盖范围

基础输入输出与 STL、二分、差分、单调结构、搜索与图遍历、并查集、筛法、背包 / LIS / 状压 / 树形 DP、KMP、随机异或哈希、线性基、贡献法、位运算及数值 / 几何标准库技巧。

版本线索：最新一批 XCPC 源文件集中在 2026-10-25 至 2026-10-28；新近整理笔记重点覆盖 KMP、图遍历和标准库。以下正文以笔记中的整理版为主，并保留其仓库来源线索。

阅读说明：ll / i64、1-based / 0-based、nxt / pi 等写法按各题原代码语义使用；复制后先核对题目下标、范围和整数上界。

快速目录

01-输入输出格式化 8
setfill / setw — 补前导零 8
其他常见格式化 8
02-二分答案模板 8
区间可行性 check — 维护可达范围 8
通用二分模板 8
注意 8
03-并查集DSU 9
路径压缩（一行版） 9
基础模板 9
扩展域并查集—敌人的敌人是朋友 9
常见应用 9
04-单调栈与单调队列 9
单调栈 — 右边第一个更大的元素 9
变体 10
单调队列 — 滑动窗口最值 10
核心思想 10
05-差分数组技巧 10
一维差分 10
二维差分 10
差分 + 二分经典题 11
06-图论DFS与BFS模板 11
memset 初始化（BFS / DFS 前必备） 11
BFS — 马的遍历（8 方向 + queue） 11
关键写法 12
BFS 网格 4 方向 — 最短路径 12
DFS 洪水填充（Flood Fill）— 8 方向 12
小技巧：vis 预处理 13
DFS — 不会被淹没的岛屿 13
树上 DFS — 子树大小 & 深度 13
方向数组速查 13
07-常用常数与宏 14
INF 哨兵值 14
基础模板 14
快读（需要时用） 14
memset vs fill 14
一些常用宏/缩写 14

08-树形DP-树上括号方案数 . . . 14
问题模型 . . . 15
思路 . . . 15
叶子节点的 DP . . . 15
非叶子节点的转移 . . . 15
答案 . . . 15
代码实现（O(n□)） . . . 15
关键点 . . . 16
样例推导 . . . 16
链 1-2-3-4 . . . 16
样例 2 的树 . . . 16
来源 . . . 16
08-贡献法与乘法原理 . . . 16
实现 . . . 16
变体 . . . 17
09-BFS常见错误与调试 . . . 17
走迷宫正确实现 . . . 17
三个经典坑 . . . 17
1. 下标不匹配：1-based 坐标没减 1 . . . 17
2. 缺少边界检查 . . . 18
3. 未标记已访问 → 重复入队 . . . 18
小优化：墙预处理 . . . 18
10-质数筛法与判定 . . . 18
O(√n) 试除判定 . . . 18
欧拉线性筛 O(N) . . . 18
核心：if (i % x == 0) break . . . 19
bitset vs bool 数组 . . . 19
埃氏筛 O(N log log N) . . . 19
选择 . . . 19
11-背包DP . . . 19
01 背包 . . . 19
二维 DP . . . 19
价值变形 . . . 20
一维优化（倒序） . . . 20
完全背包 . . . 20
一维（正序） . . . 20
01 背包 vs 完全背包 . . . 20
12-线性DP-LIS与网格路径 . . . 20
LIS（最长上升子序列） . . . 21
O(n□) 基础 DP . . . 21

双向 LIS（合唱队形） . . . 21
O(n log n) 贪心 + 二分 . . . 21
最长不升子序列（导弹拦截） . . . 21
LIS 速查 . . . 22
网格路径 DP（过河卒） . . . 22
13-lower_bound-upper_bound用法 . . . 22
基本规则 . . . 22
升序序列（默认） . . . 23
降序序列（传 greater） . . . 23
LIS 系列应用（核心场景） . . . 23
最长上升子序列 LIS . . . 23
最长不下降子序列 . . . 23
最长不上升子序列 LDS . . . 23
Dilworth 定理 . . . 23
LIS 四种变体速查 . . . 24
统计区间内元素个数 . . . 24
坐标压缩 . . . 24
set 自带的 lower_bound . . . 24
在结构体数组上二分 . . . 24
易错点 . . . 24
14-线性基 . . . 24
什么是线性基 . . . 25
插入（构造线性基） . . . 25
查询最大值 . . . 25
查询最小值 . . . 25
例题：Bitwise Maximization（最大化两集合 XOR 和） . . . 25
题意 . . . 25
推导 . . . 25
代码 . . . 26
复杂度 . . . 26
线性基经典应用 . . . 26
注意事项 . . . 26
15-状压DP-排列计数 . . . 26
问题模型 . . . 27
核心思想 . . . 27
代码模板 . . . 27
例题：Lazy Shuffling . . . 27
题意 . . . 27
推导 . . . 27
代码 . . . 28
复杂度 . . . 28
经典应用 . . . 28

注意事项 . . . 28

16-stringstream用法 . . . 28

核心用法：按空格分词 . . . 29

例题：D_Mail_Institution_Codes . . . 29

场景 . . . 29

常见坑 . . . 29

其他常用操作 . . . 29

注意事项 . . . 29

17-异或与BrianKernighan . . . 30

问题 . . . 30

核心思想 . . . 30

1. 全体异或 → 得到 a ^ b . . . 30

2. Brian Kernighan：提取最右侧的 1 . . . 30

3. 按最右 1 分组 . . . 30

完整代码 . . . 30

复杂度 . . . 30

扩展 . . . 31

注意事项 . . . 31

18-C++标准库技巧 . . . 31

一、计算几何常用函数 . . . 31

1. std::complex — 用复数做 2D 计算几何 . . . 31

2. hypot(x, y, z) — 3D 距离（C++17） . . . 31

3. remainder(x, y) — 角度归一化 . . . 31

二、数值与边界处理 . . . 31

1. std::clamp(val, low, high)（C++17） . . . 31

2. std::lerp(a, b, t)（线性插值，C++20） . . . 32

3. std::midpoint(a, b)（安全求中点，C++20） . . . 32

三、数论与位运算 . . . 32

1. std::gcd 与 std::lcm（C++17） . . . 32

2. 内置位运算（GCC 内建函数 / C++20 ） . . . 32

常用函数学习顺序 . . . 32

注意事项 . . . 32

19-取模等式计数-CRT与打表找规律 . . . 32

问题模型 . . . 33

核心思想 . . . 33

推导 . . . 33

打表找规律（暴力代码） . . . 33

正解代码 . . . 34

复杂度 . . . 34

经典应用 . . . 34

注意事项 . . . 34

验证样例 . . . 34

20-随机异或哈希-排列前缀集合 . . . 34

1. 适用条件 . . . 34
2. 核心写法 . . . 35
3. 前缀如何 O(1) 更新 . . . 35
4. 完整整理版（C++17） . . . 35
5. 碰撞概率与双哈希 . . . 36
6. 常见坑与扩展 . . . 36

21-KMP模板与题型总结 . . . 37

1. 题目与代码导航 . . . 37
2. 先统一 nxt 的含义 . . . 37
   - nxt 预处理模板 . . . 38
   - 与另一种前缀函数写法的对应 . . . 38
3. 普通匹配：P3375 . . . 38
4. 循环移位：Shift String . . . 38
5. 周期：P4391 无线传输 . . . 39
6. 数列整体平移：差分 KMP . . . 39
   - CF471D：消去整体加法 . . . 39
   - Harmonious subarrays：多次区间询问 . . . 39
7. 删除模式串：P4824 的状态回滚 . . . 39
8. KMP 自动机：把回退预处理掉 . . . 40
   - CF291E：树上继承状态 . . . 40
   - LeetCode 1397：禁止子串的双边界 DP . . . 40
9. 做题时的速查顺序 . . . 41

赛场复用检查 . . . 42

仓库代码导航

本册正文按算法主题整理。要查看完整题目语境或原始版本，可按下面的路径回到仓库搜索。较新文件用于同类写法的优先参考。

主题	仓库线索
基础、STL、搜索、DP、数学	笔记/ 中 01-21 主题笔记，详见正文各节来源
BFS / DFS 题目实现	study/ 下 P1443、P1596、P1162、CF_1037_D 等；比赛目录内亦有网格与树题
字符串匹配	study/KMP/；2026-10-21 更新的 KMP 题型总结覆盖普通匹配、差分、删除和自动机
哈希	study/stringHash/；2026-10-21 随机异或哈希笔记及 competition/2026CCPCNET/补题/eHash.cpp
XCPC 近期题目	XCPC/202AsiaXi'an Regional Contest/（2026-10-27 至 28）及 XCPC/2025Universal Cup. Extra Stage 2 Wuhan/（2026-10-25 至 26）
训练赛 / 竞赛实现	competition/ 按比赛目录组织；以题目名检索，结合原题面判断能否复用

源文件与时间说明

文件路径和题目名用于回查，不能替代题面。仓库同时存在 copy、临时文件和同步冲突文件；本册采用已整理的非冲突笔记作为主要代码载体。若较新代码只是题目实现而非通用片段，正文不会把它包装成可直接套用的模板。

01-输入输出格式化

来源：笔记/01-输入输出格式化.md | 文件修改时间：2026-05-19

setfill / setw — 补前导零

```
cout << setfill('0')          // 补什么字符（默认空格）
     << setw(2) << hours << ':'
     << setw(2) << minutes << ':'
     << setw(2) << ss << endl;
```

- ☐ setw(n) — 设置下一个输出的最小宽度，只对紧跟的一个输出有效
- ☐ setfill(c) — 宽度不足时填充的字符，一直有效直到被修改
- ☐ 头文件：#include <iomanip>（但 bits/stdc++.h 已包含）

示例：hours=3, minutes=5, ss=9 → 输出 03:05:09

其他常见格式化

```
cout << fixed << setprecision(2) << x;          // 保留两位小数
cout << hex << x;                               // 十六进制输出
cout << oct << x;                               // 八进制输出
cout << showpos << x;                           // 正数带 + 号
cout << boolalpha << flag;                      // true/false 而不是 1/0
cout << scientific << x;                        // 科学计数法
```

02-二分答案模板

来源：笔记/02-二分答案模板.md | 文件修改时间：2026-05-19

适用场景：求"最大值最小"或"最小值最大"，且答案有单调性。

区间可行性 check — 维护可达范围

从 D_Jumping_Through_Segments 提炼：

```
bool check(int k) {
    int L = 0, R = 0;          // 当前可达区间 [L, R]
    for (int i = 1; i <= n; i++) {
        L -= k;  R += k;        // 每步最多走 k，区间扩大
        if (L > r[i] || R < l[i]) return false;   // 和目标区间无交集
        L = max(L, l[i]);       // 取交集
        R = min(R, r[i]);
    }
    return true;
}
```

通用二分模板

```
int l = 0, r = INF, ans = -1;
while (l <= r) {
    int mid = (l + r) / 2;
    if (check(mid)) {
        ans = mid;       // mid 可行，尝试更小的（或更大的）
        r = mid - 1;     // 求最小可行解
        // l = mid + 1;  // 求最大可行解
    } else {
        l = mid + 1;
        // r = mid - 1;
    }
}
```

注意

- ☐ 实数二分用 for(int i=0;i<100;i++) 或 while(r-l>1e-7)
- ☐ 注意 mid 是否会溢出：mid = l + (r - l) / 2

03-并查集DSU

来源：笔记/03-并查集DSU.md | 文件修改时间：2026-05-19

路径压缩（一行版）

```
int find(int x) {
    return f[x] == x ? x : f[x] = find(f[x]);
}
```

基础模板

```
vector<int> f(N);
iota(f.begin(), f.end(), 0);   // 初始化 f[i] = i

void merge(int x, int y) {
    f[find(x)] = find(y);
}

bool same(int x, int y) {
    return find(x) == find(y);
}
```

扩展域并查集—敌人的敌人是朋友

从 K_团伙 提炼：

```
// 1~n 是自己的域，n+1~2n 是敌人的域
for (int i = 1; i <= 2 * n; i++) f[i] = i;

if (c == 'E') {            // 敌人关系
    merge(x + n, y);       // x 的敌人和 y 同类
    merge(y + n, x);       // y 的敌人和 x 同类
} else {                   // 朋友关系
    merge(x, y);
}
```

常见应用

场景	做法
判环（图中有没有多余边）	合并前 find 相同 → 多余边
连通块数量	统计 `f[i] == i` 的个数
带权 DSU（到根距离）	`dis[x] += dis[f[x]]` 路径压缩时维护
可撤销 DSU	按秩合并 + 栈记录操作

04-单调栈与单调队列

来源：笔记/04-单调栈与单调队列.md | 文件修改时间：2026-05-19

单调栈 — 右边第一个更大的元素

从 P_5788 提炼：

```
stack<int> s;
for (int i = n; i >= 1; i--) {
    while (!s.empty() && a[i] >= a[s.top()]) s.pop();
    ans[i] = s.empty() ? 0 : s.top();
    s.push(i);
}
```

变体

需求	遍历方向	比较符号
右边第一个更大	逆序	`a[i] >= a[s.top()]`
右边第一个更小	逆序	`a[i] <= a[s.top()]`
左边第一个更大	正序	`a[i] >= a[s.top()]`
左边第一个更小	正序	`a[i] <= a[s.top()]`

单调队列 — 滑动窗口最值

从 P_1886 提炼：

```
// 窗口最小值
deque<int> q;
for (int i = 0; i < n; i++) {
    if (!q.empty() && q.front() <= i - k) q.pop_front();   // 移出窗口
    while (!q.empty() && a[q.back()] >= a[i]) q.pop_back(); // 维护单调递增
    q.push_back(i);
    if (i >= k - 1) cout << a[q.front()] << " ";
}
```

求最大值只需把 `>=` 改成 `<=`（维护单调递减）。

核心思想

- [ ] deque 存的是下标不是值，方便判断是否在窗口内
- [ ] 每次：

队头出窗口 → pop_front

队尾破坏单调性 → pop_back

入队

窗口满了 → 取队头

05-差分数组技巧

来源：笔记/05-差分数组技巧.md | 文件修改时间：2026-05-19

一维差分

区间 `[l, r]` 统一加 `val` 的 O(1) 操作：

```
diff[l] += val;
diff[r + 1] -= val;
```

最后求一遍前缀和还原原数组：

```
for (int i = 1; i <= n; i++) {
    diff[i] += diff[i - 1];
    a[i] = diff[i];    // 或直接输出 diff[i]
}
```

二维差分

子矩阵 `(x1, y1)` 到 `(x2, y2)` 加 `val`：

```
diff[x1][y1] += val;
diff[x1][y2 + 1] -= val;
diff[x2 + 1][y1] -= val;
diff[x2 + 1][y2 + 1] += val;
```

还原时做二维前缀和：

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        diff[i][j] += diff[i - 1][j] + diff[i][j - 1] - diff[i - 1][j - 1];
```

差分 + 二分经典题

C_租借教室：二分订单数 check 函数中做差分判断是否够用。

Meximum_Array_2/Meximum_Array_2.cpp)：用两个差分数组分别维护两种操作区间。

06-图论DFS与BFS模板

来源：笔记/06-图论DFS与BFS模板.md | 文件修改时间：2026-10-21

memset 初始化（BFS / DFS 前必备）

从 P_1443 马的遍历 提炼：

```
bool vis[405][405];
int dis[405][405];

memset(vis, 0, sizeof(vis));    // 全部标记未访问
memset(dis, -1, sizeof(dis));   // 距离初始化为 -1（表示未到达）
```

memset 按字节填充，只适用于 0 / -1 / 0x3f 等。vector 用 fill 或 assign。

BFS — 马的遍历（8 方向 + queue<pair>）

从 P_1443 马的遍历 提炼：

```
int n, m, x, y;
int dx[] = {1, 2, 2, 1, -1, -2, -2, -1};   // 马走日
int dy[] = {-2, -1, 1, 2, 2, 1, -1, -2};
bool vis[405][405];
int dis[405][405];
queue<pair<int, int>> q;

void solve() {
    cin >> n >> m >> x >> y;
    x--; y--;                                   // 转换为 0-based
    memset(vis, 0, sizeof(vis));
    memset(dis, -1, sizeof(dis));
    dis[x][y] = 0;
    vis[x][y] = 1;
    q.push({x, y});

    while (!q.empty()) {
        int xx = q.front().first;               // pair 取值方式
        int yy = q.front().second;
        q.pop();
        for (int i = 0; i < 8; i++) {
            int x2 = xx + dx[i];                // 用 x2/y2 避免和外层变量混淆
            int y2 = yy + dy[i];
            if (x2 < 0 || x2 >= n || y2 < 0 || y2 >= m) continue;  // 边界判断
            if (!vis[x2][y2]) {
                vis[x2][y2] = 1;
                dis[x2][y2] = dis[xx][yy] + 1;
                q.push({x2, y2});
            }
        }
    }
    // 输出 dis 矩阵，-1 表示不可达
}
```

关键写法

要点	写法						
pair 队列	queue<pair<int, int>> q → q.push({x, y})						
取首坐标	q.front().first / q.front().second						
memset 初值	-1 表示不可达，0 表示未访问						
边界判断	`x2 < 0 |	|	x2 >= n |	|	y2 < 0 |	|	y2 >= m`
避免变量遮蔽	用 x2/y2 而非内层重名 x/y						

BFS 网格 4 方向 — 最短路径

从 C_幅優先探索 提炼：

```
int dx[] = {1, -1, 0, 0};
int dy[] = {0, 0, 1, -1};
queue<int> qx, qy;        // 双队列存坐标（老写法）

qx.push(sx); qy.push(sy);
while (qx.size()) {
    for (int i = 0; i < 4; i++) {
        int x = qx.front() + dx[i];
        int y = qy.front() + dy[i];
        if (x >= 1 && x <= n && y >= 1 && y <= m && 可通行) {
            qx.push(x); qy.push(y);
            dist[x][y] = dist[qx.front()][qy.front()] + 1;
            标记已访问;
        }
    }
    qx.pop(); qy.pop();
}
```

queue<pair<int,int>> 可以将横纵坐标保存在同一队列中，避免双队列同步维护。

DFS 洪水填充（Flood Fill）— 8 方向

从 P_1596 Lake Counting 提炼：

数连通块个数：每个未访问的 'W' 触发一次 DFS，DFS 内遍历 8 方向传播。

```
int n, m;
int dx[] = {-1, 0, 1, 0, 1, 1, -1, -1};  // 8 方向
int dy[] = {0, 1, 0, -1, 1, -1, 1, -1};
char a[101][101];
bool vis[101][101];
int ans = 0;

void dfs(int x, int y) {
    vis[x][y] = 1;
    for (int i = 0; i < 8; i++) {
        int nx = x + dx[i], ny = y + dy[i];
        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
        if (a[nx][ny] == 'W' && !vis[nx][ny]) {
            dfs(nx, ny);
        }
    }
}
// 主函数中：
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        if (a[i][j] == 'W' && !vis[i][j]) {
            ans++;
            dfs(i, j);
        }
```

小技巧：vis 预处理

```
// 陆地 '.' 直接标记为已访问，只在 'W' 上 DFS
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) {
        cin >> a[i][j];
        if (a[i][j] == '.') vis[i][j] = 1;
    }
```

DFS — 不会被淹没的岛屿

从 蓝桥云课 全球变暖 提炼：

问题：四面环海的 # 会被淹没，统计不会被完全淹没的岛屿数。

核心思路：DFS 遍历岛屿时，检查是否存在四面都不是 . 的 #（即不会被淹没的地块）。

```
int dx[] = {-1, 0, 1, 0};
int dy[] = {0, 1, 0, -1};
bool gd = false;  // 全局标记：当前岛屿是否有不会被淹没的地块

void dfs(int x, int y) {
    vis[x][y] = 1;
    // 检查当前格子是否四面都不是水
    bool flag = false;
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i], ny = y + dy[i];
        if (nx >= 0 && nx < n && ny >= 0 && ny < n) {
            if (a[nx][ny] == '.') flag = true;   // 有邻水 → 会被淹没
        }
    }
    if (!flag) gd = true;        // 四边没有水 → 不会淹没

    // 继续 DFS 遍历岛屿
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i], ny = y + dy[i];
        if (nx < 0 || nx >= n || ny < 0 || ny >= n) continue;
        if (!vis[nx][ny] && a[nx][ny] == '#') dfs(nx, ny);
    }
}
// 计数：只有 gd==false 的岛屿计入答案
```

树上 DFS — 子树大小 & 深度

从 P_9872_DFS_Order 和 C_Minimum_Steiner_Tree 提炼：

```
vector<int> e[N];
int siz[N], dep[N];

void dfs(int u, int fa) {
    dep[u] = dep[fa] + 1;
    siz[u] = 1;
    for (auto v : e[u]) {
        if (v == fa) continue;
        dfs(v, u);
        siz[u] += siz[v];
    }
}
// 调用：dfs(1, 0);
```

方向数组速查

场景	dx	dy
4 方向（上下左右）	{1, -1, 0, 0}	{0, 0, 1, -1}
8 方向（含对角线）	{-1, 0, 1, 0, 1, 1, -1, -1}	{0, 1, 0, -1, 1, -1, 1, -1}
马走日	{1, 2, 2, 1, -1, -2, -2, -1}	{-2, -1, 1, 2, 2, 1, -1, -2}

07-常用常数与宏

来源：笔记/07-常用常数与宏.md | 文件修改时间：2026-05-19

INF 哨兵值

```
const int INF = 0x3f3f3f3f;             // ≈ 1e9，够大且不会溢出
const int INF = 0x7f7f7f7f;             // 更大 ≈ 2e9
const long long INFLL = 0x3f3f3f3f3f3f3f3f;  // LL 版本
int maxx = -0x7f7f7f7f;                 // 求最大值初始值
```

- [ ] 0x3f3f3f3f 的优点：两个相加不超 INT_MAX，memset 能用
- [ ] 0x7f7f7f7f 更大但两个相加会溢出

基础模板

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
#define int long long

void solve() {
    // ...
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int _ = 1;
    // cin >> _;
    while (_--) {
        solve();
    }
    return 0;
}
```

快读（需要时用）

```
int read() {
    int x = 0, f = 1; char c = getchar();
    while (c < '0' || c > '9') { if (c == '-') f = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = getchar(); }
    return x * f;
}
```

memset vs fill

```
memset(arr, 0x3f, sizeof(arr));      // 初始化为 INF（按字节）
fill(arr, arr + n, 0);               // 任意值
fill(vec.begin(), vec.end(), -1);     // vector
```

一些常用宏/缩写

```
using i64 = long long;
#define all(x) x.begin(), x.end()
#define sz(x) (int)x.size()
#define pb push_back
```

08-树形DP-树上括号方案数

来源：笔记/08-树形DP-树上括号方案数.md | 文件修改时间：2026-05-23

问题模型

给一棵有根树（根为 1），每个节点赋值为 (或)，要求根到每个叶子的路径构成合法括号串。求方案数模 MOD。

思路

关注每个节点处的括号平衡度（未匹配的 (数量）。

定义 dp[u][i] = 平衡度为 i 时，子树 u 内的合法赋值方案数。

这里的"平衡度 i"指：进入节点 u 之前还有多少个未匹配的左括号。

叶子节点的 DP

叶子到达后必须把括号闭合，所以进入叶子时平衡度必须恰好为 1，叶子赋)：

```
dp[leaf][1] = 1
dp[leaf][i] = 0   (i ≠ 1)
```

非叶子节点的转移

考虑节点 u：

- ☐ 如果 u 赋 (，平衡度变为 i+1，所有子节点继承平衡度 i+1
- ☐ 子节点方案独立，乘起来：a = ∏ dp[v][i+1]
- ☐ 如果 u 赋)，平衡度变为 i-1，子节点继承平衡度 i-1（要求 i > 0，不能关闭不存在的括号）
- ☐ b = ∏ dp[v][i-1]（仅在 i > 0 时）

```
dp[u][i] = a + (i > 0 ? b : 0)
```

答案

根进入前平衡度为 0，所以答案是 dp[1][0]（根只能是 ()。

代码实现（O(n□)）

```
const int N = 3050;
const int MOD = 998244353;
vector<int> G[N];
int dp[N][N];
int n;

void dfs(int u, int fa) {
    // 叶子
    if (G[u].size() == 1 && fa != 0) {
        dp[u][1] = 1;
        return;
    }

    for (auto v : G[u]) {
        if (v == fa) continue;
        dfs(v, u);
    }

    for (int i = 0; i < n; i++) {
        int a = 1, b = 1;
        for (auto v : G[u]) {
            if (v == fa) continue;
            a = a * dp[v][i + 1] % MOD;
            if (i > 0) {
                b = b * dp[v][i - 1] % MOD;
            }
        }
        if (i == 0) {
            dp[u][i] = a % MOD;
        } else {
            dp[u][i] = (a + b) % MOD;
        }
    }
}
```

关键点

要点	说明
状态设计	以进入节点前的平衡度划分，而不是节点内的平衡度
叶子特判	只有 `dp[leaf][1] = 1`，进入时必须已有 1 个未匹配的 `(`
子节点独立	乘法原理，子节点方案互不影响
根固定	根不能是 `)`，所以只加 `a` 不加 `b`
复杂度	O(n□)，n ≤ 3000 可过

样例推导

链 1-2-3-4

```
dp[4][1] = 1
dp[3][0] = dp[4][1] = 1      → 节点3是'('
dp[3][2] = dp[4][1] = 1      → 节点3是')', i=2→1→0
dp[2][1] = dp[3][2] + dp[3][0] = 2  → 两种: (())、()()
dp[1][0] = dp[2][1] = 2      → 答案 = 2
```

对应两种方案：`(())` 和 `()()`。

样例 2 的树

根 1 有三个子树，每个子树独立，最终 `dp[1][0] = 2 × 2 × 1 = 4`。

来源

MC0560 国士知遇豫让心

08-贡献法与乘法原理

来源：笔记/08-贡献法与乘法原理.md | 文件修改时间：2026-05-20

问题是给一个字符串 s，求所有子串中不同字符个数的总和。

暴力 O(n^2) 枚举所有子串肯定不行。这类统计问题常用贡献法——不关注子串本身，而是算每个字符在多少个子串里算数。

关键：每个字符只在它第一次出现的位置贡献。同一个字符在后面再出现，前面的子串已经统计过它了，不能重复算。

这样就把问题转成了：对于位置 i 的字符，它能作为"第一次出现的字符"参与到哪些子串中？

左边界可以选 `(last[c], i]` 之间的任意位置，共 `i - last[c]` 种。

右边界可以选 `[i, n)` 之间的任意位置，共 `n - i` 种。

两边独立，乘起来就是该字符贡献的子串数。

```
sum += (i - last[c]) * (n - i)
```

实现

字符串前面加个空格变 1-indexed，避免处理边界时减出负数。last[26] 记录每个字符上次出现的位置，初始 0。

```
void solve() {
    string s;
    cin >> s;
    s = " " + s;
    int last[26] = {0};
    int sum = 0;
    for (int i = 1; i < s.size(); i++) {
        int c = s[i] - 'a';
        sum += (i - last[c]) * (s.size() - i);
        last[c] = i;
    }
    cout << sum << endl;
}
```

O(n) 时间，O(1) 额外空间。

变体

贡献法 + 乘法原理的模式很多题都能用。比如 LeetCode 907（子数组最小值之和），对每个元素算它作为最小值的子数组个数，也是左右边界独立选，乘起来就是贡献。

核心就一句话：不要枚举集合，转而算每个元素在哪些集合里是"特殊的那个"，左右边界独立选 → 乘法原理直接算。

09-BFS常见错误与调试

来源：笔记/09-BFS常见错误与调试.md | 文件修改时间：2026-05-29

来源：蓝桥云课 走迷宫

走迷宫正确实现

```
int n, m;
int dx[] = {1, 0, -1, 0};
int dy[] = {0, 1, 0, -1};
int G[300][300], vis[300][300], dist[300][300];

void solve() {
    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> G[i][j];
            if (G[i][j] == 0) vis[i][j] = 1;   // 墙直接当已访问
        }
    }
    int x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    x1--; y1--; x2--; y2--;                    // !! 题目 1-based → 0-based

    queue<pair<int, int>> q;
    q.push({x1, y1});
    dist[x1][y1] = 0;
    vis[x1][y1] = 1;

    while (!q.empty()) {
        int xx = q.front().first, yy = q.front().second;
        q.pop();
        for (int i = 0; i < 4; i++) {
            int xx2 = xx + dx[i];
            int yy2 = yy + dy[i];
            if (xx2 < 0 || xx2 >= n || yy2 < 0 || yy2 >= m) continue;  // !! 边界
            if (!vis[xx2][yy2]) {
                dist[xx2][yy2] = dist[xx][yy] + 1;
                if (xx2 == x2 && yy2 == y2) {
                    cout << dist[xx2][yy2] << endl;
                    return;
                }
                q.push({xx2, yy2});
                vis[xx2][yy2] = 1;              // !! 入队即标记
            }
        }
    }
    cout << -1 << endl;
}
```

三个经典坑

1. 下标不匹配：1-based 坐标没减 1

输入用 0 到 n-1 存地图，但题目给的起点终点是 1-based。

现象：数组越界、运行错误。

错误：`x1, y1` 直接入队，没有 `--`。

改正：读完坐标立刻减一。

```
cin >> x1 >> y1 >> x2 >> y2;
x1--; y1--; x2--; y2--;
```

2. 缺少边界检查

BFS 向四周扩展时如果不判边界，会把越界坐标压进队列，之后取 `G[xx2][yy2]` 时访问未定义内存。

```
// 错误：没用 continue 筛掉越界的
for (int i = 0; i < 4; i++) {
    int xx2 = xx + dx[i], yy2 = yy + dy[i];
    if (!vis[xx2][yy2]) { ... }   // xx2 可能是 -1 或 n
}

// 正确：先判边界
if (xx2 < 0 || xx2 >= n || yy2 < 0 || yy2 >= m) continue;
```

3. 未标记已访问 → 重复入队

入队时不立刻标 `vis`，同一个格子会被反复压进队列，复杂度爆炸。

```
// 错误：先 push 后标 vis（顺序其实没错，但有人 push 完忘了写 vis）
q.push({xx2, yy2});
// vis[xx2][yy2] = 1;   ← 漏了这一行

// 正确习惯：push 和 vis 紧挨着写
q.push({xx2, yy2});
vis[xx2][yy2] = 1;
```

小优化：墙预处理

读地图时直接把墙 `0` 标为已访问，后续 BFS 无需额外判断 `a[i][j]`。

```
if (G[i][j] == 0) vis[i][j] = 1;
```

这样扩展时只需 `if (!vis[xx2][yy2])`，不用再 `&& G[xx2][yy2] == 1`。

10-质数筛法与判定

来源：笔记/10-质数筛法与判定.md | 文件修改时间：2026-05-29

来源：P_1036_NOIP_2002_普及组_选数

O(√n) 试除判定

单个数判质数，循环到 `i*i <= x` 即可。

```
bool isPrime(int x) {
    if (x < 2) return 0;
    for (int i = 2; i * i <= x; i++) {
        if (x % i == 0) return 0;
    }
    return 1;
}
```

注意 `i*i` 可能爆 int，用 `i64` 更安全：

```
bool isPrime(i64 x) {
    if (x < 2) return 0;
    for (i64 i = 2; i * i <= x; i++) {
        if (x % i == 0) return 0;
    }
    return 1;
}
```

欧拉线性筛 O(N)

用 `bitset<N>` 当作 bool 数组标记合数，只被最小质因子筛掉，每个合数只被筛一次。

```
const int N = 100000008;
bitset<N> c;          // c[i] == 1 表示 i 是质数

c.set();              // 初始全标 1
c[0] = c[1] = 0;

vector<int> primes;
for (int i = 2; i < N; i++) {
    if (c[i]) primes.push_back(i);
    for (auto x : primes) {
        if (x * i >= N) break;
        c[x * i] = 0;
        if (i % x == 0) break;    // 关键：保证 x 是合数 i*x 的最小质因子
    }
}
```

之后判断质数直接用 c[sum] O(1) 查表，比每次 isPrime(sum) 快很多。

核心：if (i % x == 0) break

- ☐ i 是 x 的倍数时，x 已经是 i 的最小质因子
- ☐ 更大的质数 x' 筛 i*x' 时，x 才是真正的最小质因子（因为 x < x' 且 x | i）
- ☐ 加这行 break 保证每个合数只被最小质因子筛一次，复杂度 O(N)

bitset vs bool 数组

bitset<N> 每元素占 1 bit，bool 数组占 1 byte（8 bit）。N=1e8 时 bitset 只用 ~12MB，bool 数组 ~100MB。质数筛常用 bitset 省内存。

埃氏筛 O(N log log N)

简单版：把每个质数的倍数全标掉。

```
bitset<N> is_prime;
is_prime.set();
is_prime[0] = is_prime[1] = 0;
for (int i = 2; i * i <= N; i++) {
    if (is_prime[i]) {
        for (int j = i * i; j <= N; j += i) {
            is_prime[j] = 0;
        }
    }
}
```

效率比欧拉筛略差但代码更短，N 不大时足够用。

选择

方法	单次判定	多次判定	内存
isPrime() 试除	O(√n)	O(Q√n)，Q 大时炸	O(1)
埃氏筛	O(N log log N) 预处理	O(1) 查表	~12MB(bitset)
欧拉线性筛	O(N) 预处理	O(1) 查表	~12MB(bitset)

结论：多次判质数先跑筛，单次用试除。

11-背包DP

来源：笔记/11-背包DP.md | 文件修改时间：2026-06-05

来源：P_1048_NOIP_2005_普及组_采药 / P_1060_NOIP_2006_普及组_开心的金明 / P_1616_疯狂的采药

01 背包

每件物品只能选 0 或 1 次。

二维 DP

dp[i][j] = 前 i 件物品，容量 j 时的最大价值。

```
int T, M;                    // T=容量, M=物品数
int dp[105][1005];
int w[500], v[500];          // w=重量, v=价值

for (int i = 1; i <= M; i++) {
    for (int j = 0; j <= T; j++) {
        if (j >= w[i])
            dp[i][j] = max(dp[i-1][j], dp[i-1][j-w[i]] + v[i]);
        else
            dp[i][j] = dp[i-1][j];
    }
}
cout << dp[M][T] << endl;
```

价值变形

开心的金明：价值 = 价格 × 重要度，其余和标准 01 背包完全一样。

```
dp[i][j] = max(dp[i-1][j], dp[i-1][j-v[i]] + v[i] * w[i]);
//                            重量=价格  价值=价格×重要度
```

一维优化（倒序）

去掉 i 维度，j 必须倒序遍历，保证每件物品只用一次。

```
int dp[1005];
for (int i = 1; i <= M; i++) {
    for (int j = T; j >= w[i]; j--) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
```

完全背包

每件物品可以选无限次。

一维（正序）

j 正序遍历，允许同一物品被多次选取。

```
int dp[10000007];
int t, m;                    // t=容量, m=物品数
while (m--) {
    int w, v;
    cin >> w >> v;
    for (int i = w; i <= t; i++) {
        dp[i] = max(dp[i], dp[i - w] + v);
    }
}
cout << dp[t] << endl;
```

边读边算，连数组都不用开。

01 背包 vs 完全背包

	01 背包	完全背包
选择次数	0 或 1	无限
一维遍历	j 倒序（T → w[i]）	j 正序（w[i] → T）
二维递推	dp[i-1][j-w[i]]	dp[i][j-w[i]]
代表题	P_1048 采药	P_1616 疯狂的采药

关键：倒序保证每个物品只被用一次（用的是上一行的状态），正序允许同一物品多次使用（用的是当前行刚更新的状态）。

12-线性DP-LIS与网格路径

来源：笔记/12-线性DP-LIS与网格路径.md | 文件修改时间：2026-06-05

来源：P_1091_NOIP_2004_提高组_合唱队形 / P_1020_NOIP_1999_提高组_导弹拦截 / P_1002_NOIP_2002_普及组_过河卒

LIS（最长上升子序列）

O(n□) 基础 DP

dp[i] = 以 a[i] 结尾的 LIS 长度。

```
for (int i = 0; i < n; i++) {
    dp[i] = 1;
    for (int j = 0; j < i; j++) {
        if (a[j] < a[i])
            dp[i] = max(dp[i], dp[j] + 1);
    }
}
```

双向 LIS（合唱队形）

求"先升后降"的最长子序列：正着跑一次 LIS（dp1），反着跑一次 LIS（dp2），以 i 为峰值的长度 = dp1[i] + dp2[i] − 1。

```
// 正向 LIS
for (int i = 0; i < n; i++) {
    dp1[i] = 1;
    for (int j = 0; j < i; j++)
        if (a[j] < a[i]) dp1[i] = max(dp1[i], dp1[j] + 1);
}
// 反向 LIS
for (int i = n - 1; i >= 0; i--) {
    dp2[i] = 1;
    for (int j = n - 1; j > i; j--)
        if (a[j] < a[i]) dp2[i] = max(dp2[i], dp2[j] + 1);
}
// 合并
int len = 0;
for (int i = 0; i < n; i++)
    len = max(len, dp1[i] + dp2[i] - 1);
cout << n - len << endl;   // 最少出列人数
```

O(n log n) 贪心 + 二分

维护数组 f，f[i] = 长度为 i+1 的上升子序列的最小末尾值。

```
vector<int> f;
for (auto x : a) {
    auto it = lower_bound(f.begin(), f.end(), x);
    if (it == f.end())
        f.push_back(x);
    else
        *it = x;
}
// f.size() 就是 LIS 长度
```

最长不升子序列（导弹拦截）

求 LDS（最长不升子序列），用 upper_bound + greater<int>()：

```
vector<int> f;
for (auto x : h) {
    auto it = upper_bound(f.begin(), f.end(), x, greater<int>());
    if (it == f.end())
        f.push_back(x);
    else
        *it = x;
}
cout << f.size() << endl;   // 第一问：最长不升子序列长度
```

第二问（Dilworth 定理）：最少的不升子序列覆盖数 = LIS 长度。

```
vector<int> f2;
for (auto x : h) {
    auto it = lower_bound(f2.begin(), f2.end(), x);
    if (it == f2.end())
        f2.push_back(x);
    else
        *it = x;
}
cout << f2.size() << endl;   // 第二问：需要多少套系统
```

Dilworth 定理：偏序集的最少链覆盖数 = 最长反链长度。对导弹拦截来说，最少的不升子序列个数 = 最长上升子序列长度。

LIS 速查

需求	写法
最长上升（LIS）	`lower_bound(f.begin(), f.end(), x)`
最长不降	`upper_bound(f.begin(), f.end(), x)`
最长下降（LDS）	`lower_bound(f.begin(), f.end(), x, greater<int>())`
最长不升	`upper_bound(f.begin(), f.end(), x, greater<int>())`

网格路径 DP（过河卒）

从 (0,0) 走到 (n,m)，只能向右或向下，马挡在 (x,y) 及其攻击范围。

`dp[i][j]` = 到达 (i,j) 的路径数。

```
int dx[] = {2, 1, -1, -2, -2, -1, 1, 2};   // 马走日攻击范围
int dy[] = {1, 2, 2, 1, -1, -2, -2, -1};
bool block[N][N];                           // 阻挡标记

// 标记马的位置和攻击范围
block[x][y] = 1;
for (int i = 0; i < 8; i++) {
    int xx = x + dx[i], yy = y + dy[i];
    if (xx >= 0 && yy >= 0) block[xx][yy] = 1;
}

// DP
dp[0][0] = 1;
for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= m; j++) {
        if (block[i][j]) { dp[i][j] = 0; continue; }
        if (i > 0) dp[i][j] += dp[i-1][j];
        if (j > 0) dp[i][j] += dp[i][j-1];
    }
}
cout << dp[n][m] << endl;
```

递推公式：`dp[i][j] = dp[i-1][j] + dp[i][j-1]`（只能向右向下时的经典公式）。

13-lower_bound-upper_bound用法

来源：笔记/13-lower_bound-upper_bound用法.md | 文件修改时间：2026-06-05

二分查找的两个 STL 函数，有序序列上 O(log n)。

基本规则

函数	返回	条件
`lower_bound(first, last, x)`	第一个 >= x 的迭代器	升序序列
`upper_bound(first, last, x)`	第一个 > x 的迭代器	升序序列
`lower_bound(first, last, x, greater<int>())`	第一个 <= x 的迭代器	降序序列
`upper_bound(first, last, x, greater<int>())`	第一个 < x 的迭代器	降序序列

找不到时返回 `last`（end 迭代器）。

升序序列（默认）

```
vector<int> a = {1, 3, 5, 5, 7, 9};

auto it = lower_bound(a.begin(), a.end(), 5);
// *it == 5, 指向第一个 5

auto it = upper_bound(a.begin(), a.end(), 5);
// *it == 7, 指向第一个 > 5 的元素

// 统计 x 出现次数
int cnt = upper_bound(a.begin(), a.end(), x) - lower_bound(a.begin(), a.end(), x);
```

降序序列（传 greater）

```
vector<int> a = {9, 7, 5, 5, 3, 1};

auto it = lower_bound(a.begin(), a.end(), 5, greater<int>());
// *it == 5, 第一个 <= 5 的元素

auto it = upper_bound(a.begin(), a.end(), 5, greater<int>());
// *it == 3, 第一个 < 5 的元素
```

LIS 系列应用（核心场景）

最长上升子序列 LIS

```
vector<int> f;
for (auto x : a) {
    auto it = lower_bound(f.begin(), f.end(), x);
    if (it == f.end()) f.push_back(x);
    else *it = x;
}
// f.size() == LIS 长度
```

最长不下降子序列

```
auto it = upper_bound(f.begin(), f.end(), x);   // 允许相等，所以用 upper
```

最长不上升子序列 LDS

```
// 导弹拦截第一问：不升子序列
vector<int> f;
for (auto x : a) {
    auto it = upper_bound(f.begin(), f.end(), x, greater<int>());
    if (it == f.end()) f.push_back(x);
    else *it = x;
}
// f.size() == 最长不升子序列长度
```

Dilworth 定理

最少的不升子序列覆盖数 = LIS 长度：

```
// 导弹拦截第二问：最少需要多少套系统
vector<int> f;
for (auto x : a) {
    auto it = lower_bound(f.begin(), f.end(), x);   // 求 LIS
    if (it == f.end()) f.push_back(x);
    else *it = x;
}
// f.size() 就是答案
```

LIS 四种变体速查

需求	写法
最长上升 (LIS)	`lower_bound(f.begin(), f.end(), x)`
最长不降	`upper_bound(f.begin(), f.end(), x)`
最长下降 (LDS)	`lower_bound(f.begin(), f.end(), x, greater<int>())`
最长不升	`upper_bound(f.begin(), f.end(), x, greater<int>())`

来自 P_1020 导弹拦截。

统计区间内元素个数

在有序数组中统计某个值出现的次数：

```
// f.cpp — 双指针枚举子集和 + 二分计数
int l = lower_bound(q.begin(), q.end(), tar) - q.begin();
int r = upper_bound(q.begin(), q.end(), tar) - q.begin();
int cnt = r - l;           // tar 在 q 中出现的次数
```

来自 tiantisai/std/f.cpp。

坐标压缩

```
vector<int> ls;
sort(ls.begin(), ls.end());
ls.erase(unique(ls.begin(), ls.end()), ls.end());

// 把原值映射到 1-based 索引
int idx = lower_bound(ls.begin(), ls.end(), val) - ls.begin() + 1;
```

来自 MC0510 群臣功绩评定。

set 自带的 lower_bound

`set` 有自己的 `lower_bound` 成员函数，O(log n) 树上二分，比 `std::lower_bound` (O(n)) 快。

```
set<int> s;
// 找到第一个 >= l 的元素，逐个删除 [l, r] 区间
auto it = s.lower_bound(l);
while (it != s.end() && *it <= r) {
    a[*it] = x;
    it = s.erase(it);       // erase 返回下一个迭代器
}
```

来自 MC0448 诗词难度系数。

在结构体数组上二分

```
// 按 sum = x + w 排序后，二分找第一个 x - w 的位置
int idx = lower_bound(a.begin(), a.end(), array<int, 3>{x - w, INF, INF}) - a.begin();
```

来自 tiantisai/std/h.cpp。

易错点

必须有序 — 升序用默认比较器，降序必须传 `greater<int>()`

找不到返回 end — 用前检查 `it != v.end()`

获取下标 — `lower_bound(...) - v.begin()`

set 不要用 std::lower_bound — `std::lower_bound(s.begin(), s.end(), x)` 是 O(n)，要用 `s.lower_bound(x)` O(log n)

14-线性基

来源：笔记/14-线性基.md | 文件修改时间：2026-07-22

来源：Bitwise_Maximization.cpp — 2025牛客多校第二场 B

什么是线性基

线性基是 一组能张成整个 XOR 空间的基向量。给定一个集合，它的线性基满足：

- ☐ 原集合的任意子集 XOR 和，都能由线性基中若干元素的 XOR 表示
- ☐ 线性基的任意非空子集 XOR 和 不为 0
- ☐ 线性基是满足以上条件的最小集合（元素个数 = 向量空间的维数）

通常用一个 `vector<int> xxj(bit, 0)` 数组实现，`xxj[i]` 存储最高位为第 i 位的基向量。

插入（构造线性基）

将元素 `x` 插入线性基，从高位到低位尝试：

```
vector<i64> xxj(63, 0);

void insert(i64 x) {
    for (int i = 60; i >= 0; i--) {
        if ((x >> i) & 1) {
            if (xxj[i] == 0) {
                xxj[i] = x;
                break;
            } else {
                x ^= xxj[i];
            }
        }
    }
}
```

核心思想：如果当前位没有基向量，就把 `x` 存进去；否则用已有的基向量消去 `x` 的这一位，继续往下。

查询最大值

在线性基中，求能 XOR 出的最大值：

```
i64 query_max() {
    i64 res = 0;
    for (int i = 60; i >= 0; i--) {
        res = max(res, res ^ xxj[i]);
    }
    return res;
}
```

等价写法（贪心）：从高位到低位，如果异或后变大就异或。

查询最小值

线性基中最小的非零元素，就是最小的非零 `xxj[i]`（因为所有基向量的最高位互不相同，最小的那个就是 XOR 能得出的最小非零值）。

例题：Bitwise Maximization（最大化两集合 XOR 和）

题意

给定数组 `a[1..n]`，把每个数分入两个多重集合，记它们的 XOR 分别为 X 和 Y，求 X + Y 的最大值。

推导

记全部数的 XOR 为 S，则 X ⊕ Y = S

加法与 XOR 的关系：X + Y = (X ⊕ Y) + 2(X & Y) = S + 2(X & Y)

要最大化 X + Y → 最大化 X & Y

X & Y 的某位能为 1 的前提是 S 的这位为 0（若 S 某位为 1，则 X,Y 中恰有一个为 1，AND 得 0）

构造 `mask = ~S`，只保留 S 中为 0 的位

对每个 `a[i] & mask` 建立线性基

从线性基中求出最大 XOR 值 M —— 这就是能得到的最大 X & Y

答案：ans = S + 2 × M

代码

```
void solve() {
    int n;
    cin >> n;
    int a[n];
    int S = 0;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        S ^= a[i];
    }

    int mask = ~S;                      // 只保留 S 中为 0 的位
    vector<int> xxj(63, 0);

    for (int i = 0; i < n; i++) {
        int x = a[i] & mask;            // 只考虑可能贡献到 X&Y 的位
        for (int j = 30; j >= 0; j--) {  // a[i] < 2^30
            if ((x >> j) & 1) {
                if (xxj[j] == 0) {
                    xxj[j] = x;
                    break;
                } else {
                    x ^= xxj[j];
                }
            }
        }
    }

    int M = 0;
    for (int i = 30; i >= 0; i--) {
        M = max(M, M ^ xxj[i]);
    }

    int ans = S + 2LL * M;
    cout << ans << endl;
}
```

复杂度

- ☐ 插入 O(n log A)，查询 O(log A)
- ☐ 本题 n 总和 ≤ 5×10□，A < 2□□

线性基经典应用

问题	做法
最大 XOR 子集	建线性基，query_max()
最小 XOR 子集（非零）	建线性基，找最小的非零 xxj[i]
第 k 小 XOR 子集	线性基高斯消元成最简形式，按位选
判断一个数能否被表示	插入过程中看是否被消成 0
两集合 XOR 和最大化	本题，S + 2 × query_max_on(a[i] & ~S)

注意事项

- ☐ 开 63 是基于 64 位整数（long long）；若值域 < 2□□，开 30 够用
- ☐ 线性基的数组长度 = 值域的二进制位数，不是 n
- ☐ 插入时 xxj[i] 存储的是 最高位为 i 的基向量，低位可能有其他位的信息
- ☐ 线性基可以 高斯消元化成最简形式（每行除最高位外只保留一个 1），用于求第 k 小
- ☐ 0 无法被线性基表示（xor 空集得 0，但线性基本身不存 0）

15-状压DP-排列计数

来源：笔记/15-状压DP-排列计数.md | 文件修改时间：2026-07-23

来源：Lazy_Shuffling.cpp — 2025牛客多校第二场 L

问题模型

给定一个排列的某种约束关系（如逆序对的先后限制），统计满足所有约束的排列个数。

核心思想

用 dp[s] 表示 已经放置在末尾的元素集合为 s 时，满足约束的方案数。从空集开始，每次尝试往末尾添加一个元素，检查其所有前置条件是否已满足。

代码模板

```
dp[0] = 1;
for (int s = 1; s < (1 << n); s++) {
    int sum = 0;
    for (int u = 0; u < n; u++) {
        if (s & (1 << u)) {
            if ((pre[u] & s) == 0) {      // u 的所有前置元素都已不在 s 中（已放置）
                sum += dp[s ^ (1 << u)];
            }
        }
    }
    dp[s] = sum;
}
// dp[(1<<n)-1] = 满足约束的排列数
```

关键条件 (pre[u] & s) == 0：

- ☐ pre[u] 是必须放在 u 之前 的元素集合（位掩码）
- ☐ s 是还未放置的元素集合
- ☐ 如果 pre[u] 中的元素还在 s 里（未放置），说明有前置约束未满足，不能放 u

例题：Lazy Shuffling

题意

给定一个排列 p，允许执行某种"懒人洗牌"操作。求所有可能的操作结果排列的数量。

推导

对排列 p 统计逆序对，同时记录每个元素的"前驱"：scin[p[j]] |= (1 << p[i]) 表示 p[i] 必须在 p[j] 之前（因为 p[i] > p[j] 且 i < j，是逆序对）

若无逆序对（已有序），答案为 n!

否则用状压 DP 统计满足前置约束的排列数 dp[(1<<n)-1]

最终答案乘以 2（题目中洗牌过程的对称性）：ans = 2 × dp[(1<<n)-1]

代码

```
void solve() {
    int n;
    cin >> n;
    vector<int> p(n);
    for (int i = 0; i < n; i++) {
        cin >> p[i];
        p[i]--;
    }

    vector<int> pre(n, 0);   // pre[u] = 必须在 u 之前的元素集合
    int inv = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i] > p[j]) {
                inv++;
                pre[p[j]] |= (1 << p[i]);
            }
        }
    }

    if (inv == 0) {
        int ans = 1;
        for (int i = 1; i <= n; i++) ans = ans * i % MOD;
        cout << ans << endl;
        return;
    }

    vector<int> dp(1 << n, 0);
    dp[0] = 1;
    for (int s = 1; s < (1 << n); s++) {
        int sum = 0;
        for (int u = 0; u < n; u++) {
            if (s & (1 << u)) {
                if ((pre[u] & s) == 0) {    // u 的前置条件已满足
                    sum = (sum + dp[s ^ (1 << u)]) % MOD;
                }
            }
        }
        dp[s] = sum;
    }

    cout << 2LL * dp[(1 << n) - 1] % MOD << endl;
}
```

复杂度

- ☐ 时间 O(n × 2□)，n ≤ 20 左右可用
- ☐ 空间 O(2□)

经典应用

问题	做法
统计拓扑排序方案数	对 DAG 的边建立 pre[u]，跑状压 DP
有偏序的排列计数	逆序对 / 竞赛中的大小关系约束
Hamiltonian 路径计数	dp[s][v] 表示已走集合 s，当前在 v

注意事项

- ☐ n 不能太大（≤ 20 比较安全），否则 2□ 爆炸
- ☐ pre[u] 的构建要根据具体约束的语义，保证偏序关系的传递性
- ☐ DP 顺序从 1 到 (1<<n)-1，保证子集 s ^ (1<<u) 已经在之前计算过
- ☐ 模数通常取 998244353 或 1e9+7

16-stringstream用法

来源：笔记/16-stringstream用法.md | 文件修改时间：2026-10-21

来源：D_Mail_Institution_Codes.cpp — 2025牛客多校第七场 D

核心用法：按空格分词

stringstream 用流运算符 >> 提取，自动以空白字符（空格 / Tab / 换行）为分隔符切词。适合处理每行元素数量不固定的输入。

```
#include <sstream>   // 或直接 <bits/stdc++.h>

string line;
getline(cin, line);          // 读入一整行

stringstream ss(line);       // 从字符串构造流
string word;
while (ss >> word) {         // 逐个提取，直到流为空
    // 处理 word（以空白为分隔的每个单词）
}
```

例题：D_Mail_Institution_Codes

场景

每行是一串 由空格分隔的单词（行内单词个数不定，行数 n 已知），需要把每个单词分别存下来。用 stringstream 逐行分词即可。

```
void solve() {
    ll n;
    cin >> n;

    vector<vector<string>> a(n);       // a[i] 存第 i 行的所有单词
    for (ll i = 0; i < n; i++) {
        string line;
        getline(cin, line);            // 读一整行
        stringstream ss(line);         // 构造流
        string word;
        while (ss >> word) {           // 按空格切词
            a[i].push_back(word);
        }
    }

    // 后续比较每个单词的首字母，做压缩输出……
}
```

常见坑

- ☐ cin >> n 之后直接 getline 会先读到那一行的换行，得到空串。要么先 getline 消费掉残留换行，要么把 cin 换成整行读取再解析
- ☐ 上面代码中处理方式是：getline(cin, s); s = ""; 先吞掉残留换行，再开始循环读取

其他常用操作

```
stringstream ss;

// 1. 数字 → 字符串
int x = 123;
stringstream s1; s1 << x;
string str = s1.str();               // "123"

// 2. 字符串 → 数字（可处理多数字混合）
stringstream s2("456 789");
int a, b; s2 >> a >> b;              // a=456, b=789

// 3. 逐个字符 / 按自定义分隔符
// stringstream 默认按空白切，自定义分隔符需配合 getline(ss, tok, ',')
stringstream s3("a,b,c");
string tok;
while (getline(s3, tok, ',')) {      // 按逗号切
    // tok = "a" → "b" → "c"
}
```

注意事项

- ☐ stringstream 头文件 <sstream>（竞赛常用 <bits/stdc++.h> 已包含）

- [ ] ss >> x 会跳过前导空白，读到 EOF 或失败时返回 false，可用于 while 循环
- [ ] ss.clear() 与 ss.str("") 用于复用同一个流对象清空（每次构造新流更简单）
- [ ] 比 getline + 手写 split 更简洁，比 strtok 更安全（strtok 是 C 风格，会破坏原字符串、非线程安全）
- [ ] 性能：数据量极大时（如 1e6+ 行）stringstream 比手写扫描稍慢，但绝大多数场景够用

17-异或与BrianKernighan

来源：笔记/17-异或与BrianKernighan.md | 文件修改时间：2026-10-16

来源：Two_Odd_Count_Numbers.cpp — 左程云视频 030_5

问题

数组 nums 中，只有 两个数 a、b 出现奇数次，其余所有数都出现偶数次。找出 a、b。

核心思想

1. 全体异或 → 得到 a ^ b

异或的性质：x ^ x = 0，0 ^ x = x。把数组全部异或一遍，出现偶数次的数全部抵消，剩下的就是 eor1 = a ^ b。

2. Brian Kernighan：提取最右侧的 1

a != b，所以 eor1 = a ^ b != 0，至少有一位是 1。Brian Kernighan 算法一步提取最右侧的 1：

```
int rightOne = eor1 & (-eor1);
```

-eor1 是补码 = ~eor1 + 1，与 eor1 相与后，除了最右侧的那个 1，其余位全为 0。

3. 按最右 1 分组

rightOne 这一位在 a 和 b 中必然一个为 0、一个为 1（因为它们是 a^b 中为 1 的位）。把数组中该位为 0 的数全部异或，得到的就是 a（或 b）本身：

```
int eor2 = 0;
for (int num : nums) {
    if ((num & rightOne) == 0) {
        eor2 ^= num;
    }
}
// eor2 = a 或 b 中的某一个
```

另一个数 = eor1 ^ eor2。

完整代码

```
pair<int, int> twoOddNumbers(vector<int>& nums) {
    int eor1 = 0;
    for (int num : nums) {
        eor1 ^= num;              // = a ^ b
    }

    int rightOne = eor1 & (-eor1); // Brian Kernighan：最右侧的 1

    int eor2 = 0;
    for (int num : nums) {
        if ((num & rightOne) == 0) {
            eor2 ^= num;          // = a 或 b
        }
    }

    int a = eor2;
    int b = eor1 ^ eor2;
    return {a, b};
}
```

复杂度

- [ ] 时间 O(n)，空间 O(1)

扩展

变体	做法
只有一个数出现奇数次	全体异或一次即可
两个数出现奇数次（本题）	全体异或 → Brian Kernighan 分组 → 再异或
k 个不同的数出现奇数次	按二进制位统计奇偶性（每个数的二进制位分别计数 % k）

注意事项

- [ ] `a ^ b != 0` 是关键前提（a ≠ b），否则最右 1 不存在
- [ ] `rightOne` 分组的两组只有一组包含 a 或 b，另一组全是偶数次的数（会被抵消）
- [ ] 用 `int` 时 `-eor1` 即补码取负，C++ 中直接写 `eor1 & (-eor1)` 即可

18-C++标准库技巧

来源：笔记/18-C++标准库技巧.md | 文件修改时间：2026-10-21

来源：左程云系列 + 个人整理 — C++ 竞赛高频标准库函数速查

一、计算几何常用函数

1. `std::complex<double>` — 用复数做 2D 计算几何

`std::complex<double>` 可以表示二维向量，并通过复数运算完成向量计算：

```
#include <complex>
using Point = complex<double>;

Point A = {3, 2}, B = {-2, 3};
Point C = A + B;            // 向量加法
Point D = A * 2.0;          // 向量数乘

double len = abs(A);        // 模长 sqrt(x^2 + y^2)，等价于 hypot(x, y)
double angle = arg(A);      // 极角，等价于 atan2(y, x)
```

点乘与叉乘：

```
// conj(A) 是共轭复数 (x, -y)
double dot = (conj(A) * B).real();   // 向量点乘 A · B
double cross = (conj(A) * B).imag(); // 向量叉乘 A × B（判断顺逆时针）
```

向量旋转：

```
// 构造长度为 r，角度为 theta 的向量 (r * cosθ, r * sinθ)
Point P = polar(1.0, theta);
Point rotated = A * polar(1.0, PI / 2.0); // 将向量 A 逆时针旋转 90 度！
```

2. `hypot(x, y, z)` — 3D 距离（C++17）

C++17 扩展了 hypot，支持三维距离，不用手动写 `sqrt(x*x + y*y + z*z)` 防溢出：

```
double dist3d = hypot(x, y, z);
```

3. `remainder(x, y)` — 角度归一化

把角度规范化到 `[-π, π]`，只要一行：

```
double theta_norm = remainder(theta, 2 * M_PI);   // 规范到 [-PI, PI]
```

二、数值与边界处理

1. `std::clamp(val, low, high)`（C++17）

限制数值在区间内，比 `min(max(val, low), high)` 清晰很多：

```
#include <algorithm>

double val = 1.5;
double safe_cos = clamp(val, -1.0, 1.0);   // 防止 acos 因浮点误差爆 NaN
double angle = acos(safe_cos);
```

2. std::lerp(a, b, t)（线性插值，C++20）

求线段上按比例 t 分割的点（a + t(b-a)），保证 t=1 时严格等于 b，消除浮点累积误差：

```
#include <cmath>
double mid = lerp(x1, x2, 0.5);   // 精确求中点
```

3. std::midpoint(a, b)（安全求中点，C++20）

防止大整数相加溢出的中点：

```
#include <numeric>
long long mid = midpoint(L, R);   // 等价于 L + (R - L) / 2，绝不爆 long long
```

三、数论与位运算

1. std::gcd 与 std::lcm（C++17）

<numeric> 头文件，不用自己写辗转相除法：

```
#include <numeric>
long long g = gcd(a, b);
long long l = lcm(a, b);
```

2. 内置位运算（GCC 内建函数 / C++20 <bit>）

状态压缩（状压 DP）必备：

功能	GCC 内建写法	C++20 标准写法（ ）
数二进制中 1 的个数	__builtin_popcountll(x)	std::popcount(x)
求最高位 1 前面有几个 0	__builtin_clzll(x)	std::countl_zero(x)
求最低位 1 后面有几个 0	__builtin_ctzll(x)	std::countr_zero(x)

常用函数学习顺序

atan2(y, x) 与 hypot(x, y) — 几何基础

std::clamp — 防三角函数溢出爆 NaN

std::gcd / std::lcm — 数论高频

std::complex<double> — 用于多边形与向量旋转计算

注意事项

- [ ] std::complex 的 operator* 是复数乘法（模相乘、角相加），做向量数乘时用 A * 2.0（实数），不是两个 Point 直接相乘
- [ ] conj(A) * B 中的 conj 使实部 = 点乘、虚部 = 叉乘，可用于计算点乘和叉乘
- [ ] hypot 会自动防止中间平方溢出；三维 hypot(x,y,z) 需要 C++17
- [ ] lerp/midpoint 需要 C++20 编译选项
- [ ] GCC 内建函数 __builtin_* 结尾 ll 表示 long long 版本，带 u 的无符号更安全

19-取模等式计数-CRT与打表找规律

来源：笔记/19-取模等式计数-CRT与打表找规律.md | 文件修改时间：2026-10-16

来源：Bobo_s_Lucky_Modulo.cpp — 2025牛客多校第七场 L（Problem L. Bobo 的幸运取模）

问题模型

给定正整数 n，统计有序数对 (a, b)（1 ⩽ a, b ⩽ n）的个数，满足：

```
(a mod b) + 1 = a mod (b + 1)
```

核心思想

打表找规律：先暴力枚举小范围，打印每个 b 对应的合法 a，观察到——对每个固定的 b，合法 a 在模 b(b+1) 意义下是一段连续区间 [b□, b□+b−1]，且最小合法 a 是 b□。

中国剩余定理解释：因为 gcd(b, b+1) = 1，条件可写成对同一余数 r 满足 a ≡ r (mod b) 与 a ≡ r+1 (mod b+1)，CRT 保证所有 r 的解在模 b(b+1) 下恰好构成连续 b 个值——这就是"区间规律"的根源。

O(√n) 计数：b > √n 时 b□ > n，没有合法 a，只需枚举 b 到 √n，对每个 b 用整除（完整周期 + 尾部余数）统计区间内 a 的个数。

推导

设 r = a mod b（0 ⩽ r ⩽ b−1），则 a = kb + r。代入条件：

```
a mod (b+1) = (kb + r) mod (b+1) = (r − k) mod (b+1)
```

要求等于 r+1，即 (r − k) ≡ r + 1 (mod b+1)，化简得：

```
−k ≡ 1 (mod b+1)   =>   k ≡ −1 ≡ b (mod b+1)
```

所以 k = b + m(b+1)（m ⩾ 0），于是：

```
a = kb + r = b□ + m·b(b+1) + r,   0 ⩽ r ⩽ b−1
```

结论：对固定 b，合法 a 是 [b□, b□+b−1] 每个值再整体加 b(b+1) 整数倍的并集，即：

```
合法 a ≡ b□, b□+1, ..., b□+b−1   (mod b(b+1))
```

CRT 视角：条件要求 a mod b = r 且 a mod (b+1) = r+1（同一 r）。由于 b 与 b+1 互质，对每个 r 这个同余方程组在模 b(b+1) 下有唯一解，且所有 r 的解合起来正好是连续 b 个值——所以打表看到的"连续区间"不是巧合，是 CRT 的直接结论。

计数：最小合法 a = b□，故只需枚举 b*b <= n。对每个 b 令 t = b(b+1)，把 n 写成 n = cnt·t + s：

- □ 模 t 的每个完整周期内恰好包含 b 个合法余数（b□..b□+b−1）→ 完整周期 cnt 个，贡献 cnt·b
- □ 尾部余数 s 若 ⩾ b□，则 [b□, s] 里还能贡献 s − b□ + 1 个

打表找规律（暴力代码）

```
void solve() {   // 打印每个 b 对应的合法 a，观察规律
    for (int b = 1; b <= 20; b++) {
        cout << "b = " << b << ": ";
        for (int a = 1; a <= 200; a++) {
            if ((a % b) + 1 == a % (b + 1)) cout << a << " ";
        }
        cout << endl;
    }
}
```

输出节选：

```
b=1: 1 3 5 7 9 11 ...          // 奇数，模 2 余 1
b=2: 4 5 10 11 16 17 ...       // 模 6 余 4,5
b=3: 9 10 11 21 22 23 ...      // 模 12 余 9,10,11
b=4: 16 17 18 19 36 37 ...     // 模 20 余 16..19
```

观察：对 b，合法 a 模 b(b+1) 落在连续区间 [b□, b□+b−1]，且最小值为 b□ → 据此写出 O(√n) 计数。

正解代码

```
void solve() {
    ll n;
    cin >> n;
    ll ans = 0;
    for (ll b = 1; b * b <= n; b++) {   // 最小合法 a = b□, b > √n 时无解
        ll t = b * (b + 1);
        ll cnt = n / t;                 // 完整周期个数
        ans += cnt * b;                 // 每个完整周期贡献 b 个
        ll s = n % t;                   // 尾部余数
        if (s >= b * b) ans += s - b * b + 1;   // 尾部 [b□, s] 内的 a
    }
    cout << ans << endl;
}
```

复杂度

- □ 时间 O(√n)：n ≤ 10□□ 时约 10□ 次循环
- □ 空间 O(1)

经典应用

问题	做法
满足 (a mod b)+1 = a mod (b+1) 的数对计数	打表找区间规律 + CRT 证明 + O(√n) 整除计数
带模的等式计数（两同余式联立）	写成 a ≡ r (mod b)、a ≡ r+1 (mod b+1)，CRT 合并
满足某同余条件的数在 [1,n] 内个数	完整周期 + 尾部余数整除统计，模板化处理

注意事项

- □ 必须开 long long：n 最大 10□□，答案量级约 n·ln(√n) ≈ 1.4×10□□，b*b、b*(b+1)、cnt*b 都会溢出 int
- □ 循环上界用 b*b <= n（√n），因为最小合法 a = b□，b 更大时区间起点已超出 n
- □ 尾部判断用 s >= b*b（含等号）：b□ 本身是合法 a（对应 r = 0）
- □ 别对 a 暴力枚举——a 可到 10□□ 量级，只有 O(√n) 的做法可行

验证样例

- □ n = 5 → 5，五对为 (1,1) (3,1) (4,2) (5,1) (5,2)
- □ n = 17 → 20

20-随机异或哈希-排列前缀集合

来源：笔记/20-随机异或哈希-排列前缀集合.md | 文件修改时间：2026-10-21

来源：2026 CCPC 网络赛补题 · eHash.cpp

给每个值分配一个固定的 64 位随机数，再把集合中所有值的随机数异或起来，就能用一个整数表示集合的“指纹”。插入一个值只需一次异或，比较两个集合只需比较指纹。

1. 适用条件

比较对象	能否直接使用	原因
集合、排列的前缀集合	可以	每个值最多出现一次，顺序不影响集合
序列是否完全相同	不可以	异或不保留顺序
含重复值的多重集合	不可以	相同值出现两次会抵消，只保留次数的奇偶性

以下算法假设三个序列都是 1..n 的排列；用于其他输入模型时，需要先确认每个值在单个序列中至多出现一次。

2. 核心写法

```
using u64 = uint64_t;
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

vector<u64> weight(n + 1);
for (int x = 1; x <= n; ++x) weight[x] = rng();

u64 h = 0;
for (int i = 1; i <= n; ++i) {
    h ^= weight[a[i]];  // 加入“值”的权重，而不是“位置”的权重
}
```

- [ ] mt19937_64 生成 64 位伪随机数，每次调用 rng() 得到一个权重。
- [ ] 同一个值在所有序列中必须共用 weight[x]，不能各生成一套。
- [ ] 先生成权重，再重复使用；不能每次加入元素都重新调用 rng()。
- [ ] 用 uint64_t 保存哈希，避免把生成的无符号随机数存入 long long。

对于集合 S，定义：

```
H(S) = 对所有 x ∈ S 的 weight[x] 做异或
```

异或满足交换律、结合律，因此 {1, 2} 和 {2, 1} 的指纹相同。相同集合一定得到相同指纹；不同集合也可能碰撞，所以指纹相同只是高概率相同。

3. 前缀如何 O(1) 更新

pre[j] 表示第 j 个排列当前前缀的指纹。长度从 i - 1 增加到 i 时，只加入一个新值：

```
pre[j] ^= weight[a[j][i]];
```

例如 [1, 3, 2] 和 [2, 1, 3] 都得到 weight[1] ^ weight[2] ^ weight[3]。排列顺序不同，但前缀集合相同。

来源代码在每个位置检查三对前缀：

```
if (pre[0] == pre[1] || pre[1] == pre[2] || pre[0] == pre[2]) {
    ++ans;
}
```

同一个位置最多计数一次：判断的是“至少有一对相同”，不是相同的配对数量。原代码的 ans = 1 配合输出 ans - 1，可直接整理成从 0 开始计数。

4. 完整整理版（C++17）

保持来源代码的输入输出方式：先读测试组数，每组读 n 和三个长度为 n 的排列。

```
#include <bits/stdc++.h>
using namespace std;
using u64 = uint64_t;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

void solve() {
    int n;
    cin >> n;

    array<vector<int>, 3> a;
    for (auto& row : a) {
        row.resize(n + 1);
        for (int i = 1; i <= n; ++i) cin >> row[i];
    }

    vector<u64> weight(n + 1);
    for (int x = 1; x <= n; ++x) weight[x] = rng();

    array<u64, 3> pre{};
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
        for (int j = 0; j < 3; ++j) pre[j] ^= weight[a[j][i]];
        if (pre[0] == pre[1] || pre[1] == pre[2] || pre[0] == pre[2]) {
            ++ans;
        }
    }
    cout << ans << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    while (t--) solve();
    return 0;
}
```

项目	复杂度
生成权重、读入三个排列	O(n)
扫描所有前缀	O(n)，每个位置三次更新、三次比较
总空间	O(n)，保存排列与权重

整理版用 `vector` 替代原代码的 `ll b[n + 1]`，后者是变长数组，不属于标准 C++。以下模板采用标准 C++ 容器。

5. 碰撞概率与双哈希

如果将各个权重视为独立、均匀的 64 位随机数，对两个固定的不同集合，碰撞概率为 `2^-64`。每组扫描至多比较 `3n` 对，按并集界，误判概率至多为 `3n / 2^64`。

这是理想随机模型下的估计；`mt19937_64` 是伪随机生成器，实际实现仍是随机化算法，并非确定性证明。需要进一步降低风险时，可以维护两套权重和前缀指纹：

```
vector<u64> w1(n + 1), w2(n + 1);
for (int x = 1; x <= n; ++x) {
    w1[x] = rng();
    w2[x] = rng();
}
array<u64, 3> h1{}, h2{};

// 扩展第 j 个排列的前缀
h1[j] ^= w1[a[j][i]];
h2[j] ^= w2[a[j][i]];

// 两个分量都相同才认为指纹相同
bool equal = h1[u] == h1[v] && h2[u] == h2[v];
```

在两套权重独立的理想模型下，固定一对不同集合的碰撞概率降为 `2^-128`。双哈希也不能修复重复值被异或抵消的问题。

6. 常见坑与扩展

直接异或原值不是随机哈希。 `{1, 2}` 和 `{4, 7}` 都异或成 `3`，结构性碰撞很容易构造。

重复值会抵消。 `[1, 1]` 和空序列都得到 `0`；即使长度相同，`[1, 1]` 和 `[2, 2]` 也无法区分。

值域必须能作为下标。 示例假设 `1 <= a[j][i] <= n`；值域大时先统一离散化，或用映射保存权重。

每组要清空状态。 `pre{}` 在 `solve()` 内初始化，避免上一组的前缀残留。

空前缀是否计数取决于题意。 来源代码只扫描 `1..n`，不会统计长度为 `0` 的前缀。

扩展场景	写法与限制
集合插入、删除	都执行 `h ^= weight[x]`；插入前确认不存在，删除前确认存在
区间指纹	`pref[r] ^ pref[l - 1]`；有重复值时表示奇偶状态，不是多重集合
比较多重集合	可改用随机权重求和，并按 `2^64` 取模；保留次数信息，仍存在碰撞

记忆要点：按值赋权，共用权重，前缀异或；适用集合，不保留顺序和完整次数。

21-KMP模板与题型总结

来源：笔记/21-KMP模板与题型总结.md | 文件修改时间：2026-10-21

以 `nxt[0]=-1`、`j` 表示已匹配长度 为主线，整理 KMP 模板、相关题型与实现中的易错点。

1. 题目与代码导航

题目	核心模型	关键变化
P3375 · KMP 模板	找出模式串的全部出现位置	匹配后回退，允许重叠；输出 1-based 起点
Shift String	代码在 `s+s` 中寻找目标串	等长循环移位转子串匹配，返回首个 0-based 起点
P4391 · 无线传输	求最短周期长度	`n-nxt[n]`；不必整除整个串
CF471D · MUH and Cube Walls	序列整体平移后匹配	相邻差分相等；模式长度为 1 时单独处理
P4824 · Censoring	扫描时删除出现的模式串	栈保存字符和匹配状态，删除后恢复栈顶状态
CF291E · Tree String Problem	树边带字符串，统计路径上的匹配	显式栈传递状态，自动机 O(1) 转移
LeetCode 1397 · 找到所有好字符串	字典序区间内计数，禁止出现 `evil`	KMP 自动机 + 双边界数位 DP
Harmonious subarrays · 改进版	多次询问区间内有多少个平移匹配	差分 KMP 跑一次，匹配起点前缀和回答询问

2. 先统一 nxt 的含义

对长度为 `m` 的模式串 `pat`：

- [ ] `nxt[0]=-1` 是哨兵，不是实际的前后缀长度。
- [ ] 对 `1 <= len <= m`，`nxt[len]` 是 `pat[0..len-1]` 的最长相等真前后缀长度。
- [ ] “真”表示不能取整个串，所以 `nxt[len] < len`。
- [ ] 匹配状态 `j` 表示已经匹配了模式串前 `j` 个字符，下一个应比较 `pat[j]`。

例如 `pat="ababa"`：

前缀长度	0	1	2	3	4	5
前缀	空	a	ab	aba	abab	ababa
`nxt[len]`	-1	0	0	1	2	3

nxt 预处理模板

```
vector<ll> kmp_pre(const string& pat){
    ll m=pat.size();
    vector<ll> nxt(m+1);
    ll i=0,j=-1;
    nxt[0]=-1;
    while(i<m){
        while(j!=-1 && pat[i]!=pat[j]){
            j=nxt[j];
        }
        nxt[++i]=++j;
    }
    return nxt;
}
```

这里的 `nxt[++i]=++j` 表示：找到能继续接上当前字符的 border 后，前缀长度和匹配长度一起增加。若失配一直退到 `-1`，递增后就是 `0`，因此也正确处理首字符失配。

`j!=-1` 必须放在 `&&` 左边，让短路求值避免访问 `pat[-1]`。数组要开 `m+1`，因为会写到 `nxt[m]`。

与另一种前缀函数写法的对应

比赛目录中的两个区间匹配文件使用的是 `pi` 风格，虽然变量也叫 `nxt`：

项目	长度式	0-based 前缀函数
数组长度	`m+1`	`m`
初值	`nxt[0]=-1`	`pi[0]=0`
数组含义	长度为 `len` 的前缀的 border	以位置 `i` 结尾的前缀的 border
失配回退	`j=nxt[j]`	`j=pi[j-1]`，要求 `j>0`
完整匹配后回退	`j=nxt[m]`	`j=pi[m-1]`

两者满足 `nxt[len]=pi[len-1]`（`len>=1`）。不能只替换初值，保留另一套下标公式。

3. 普通匹配：P3375

下面返回全部 0-based 起点，假设模式串非空；完整程序补上常用头文件和 `#define ll long long` 即可。

```
vector<ll> kmp_match(const string& s,const string& pat){
    ll n=s.size(),m=pat.size();
    auto nxt=kmp_pre(pat);
    vector<ll> pos;
    ll i=0,j=0;
    while(i<n){
        while(j!=-1 && s[i]!=pat[j]){
            j=nxt[j];
        }
        i++;j++;
        if(j==m){
            pos.push_back(i-m);
            j=nxt[j];
        }
    }
    return pos;
}
```

- [ ] `i` 已指向匹配末尾的后一位，因此 0-based 起点是 `i-m`，P3375 输出时再加 `1`。
- [ ] 匹配成功后回退到 `nxt[m]`，保留可用后缀。例如 `aba` 在 `ababa` 中出现两次，不能直接清零漏掉重叠。
- [ ] P3375 还要求输出 `nxt[1..m]`；源码中的计数返回值不是该题最终输出的一部分。
- [ ] 预处理 $O(m)$，线性扫描 $O(n)$，保存所有匹配位置最多 $O(n)$ 空间。

4. 循环移位：Shift String

等长字符串 `s`、`t` 中，`t` 是 `s` 的某个循环移位，当且仅当它出现在 `s+s` 中。查找第一个匹配起点即可得到一个对应的左移量。

例如 `s="abcd"`、`t="cdab"`，在 `abcdabcd` 中首次出现的起点为 `2`。

当前代码的特点与前提：

用 `s1+=s1` 构造文本，找到第一次匹配后立即返回 `i-m`。

用全局 ok 标记是否找到，每组开头已执行 ok=0。也可以只根据返回值是否为 -1 判断。

如果题意是循环移位，必须保证两串原长度相同；当前代码没有主动检查等长。

返回值是左移量；如果题目问右移量，需要按题意换算，不能直接混用。

总时间和空间均为 O(n)，这里 n 是原字符串长度。

5. 周期：P4391 无线传输

设 b=nxt[n]，那么最短周期长度为：

```
ll p=n-nxt[n];
```

因为长度为 b 的前后缀相等，意味着 s[i]=s[i-p] 对 p<=i<n 成立；最长 border 对应最短周期。

“周期”与“完整重复单元”要分开：

问题	判断方式	例子
原串是某个无限重复串的前缀	直接取 p=n-nxt[n]	ababa 的最短周期是 2
原串由一个短串完整重复组成	还要求 n%p==0；若要求至少重复两次，还需 p<n	ababa 不能被 ab 完整拼成

当前 P4391 代码直接输出 n-next[n]，对应允许末尾只保留部分周期的模型。时间 O(n)，空间 O(n)。

6. 数列整体平移：差分 KMP

CF471D：消去整体加法

若一个窗口满足 a[l+i]=b[i]+c，相邻作差后常数 c 被消去：

```
a[l+i+1]-a[l+i] = b[i+1]-b[i]
```

因此对长度为 n-1 的 difa 和长度为 m-1 的 difb 做 KMP；KMP 只要求能判等，不限于字符。

- [ ] 模式原长度 m==1 时，每个位置都能通过整体平移匹配，答案是 n，不要进入空模式 KMP。
- [ ] m>n 时答案为 0。
- [ ] 当前源码把 difb 作为模式传入，difa 作为文本传入；局部变量名 s,p,nn,mm 比较容易读反。
- [ ] 当前源码的 nxt 按文本长度开辟；若允许 m>n，预处理可能越界。通用整理时应先特判 m>n，或始终按模式长度分配 nxt。
- [ ] lambda 参数可改为 const vector<ll>&，避免复制整个差分数组。

时间 O(n+m)，空间 O(n+m)。做差时也要确认原始数值范围，ll 不能保证任意输入差值都不溢出。

Harmonious subarrays：多次区间询问

仓库中有旧版和改进版 2.cpp。改进版先全局匹配，再记录每个 起点 是否匹配：

```
is_match[x] = 差分模式是否从差分文本的 0-based 位置 x 开始出现
pre[k] = is_match[0..k-1] 的和
```

询问原数组的 1-based 区间 [l,r]，原模式长度为 m：

- [ ] 区间长度小于 m：答案为 0。
- [ ] m==1：答案为 r-l+1。
- [ ] 其余情况，原数组合法起点是 [l,r-m+1]，转成差分数组 0-based 起点是 [l-1,r-m]。

所以答案为：

```
pre[r-m+1]-pre[l-1]
```

预处理 O(n+m)，每次询问 O(1)，总时间 O(n+m+q)。

旧版的教训：构造 b 的差分时误用了 i<n，应为 i<m；将数值差分强行加入 string 会窄化为字符、丢失信息；prea 是 0-based，而查询下标是 1-based。逐次重跑匹配还会重复扫描，应优先使用 2.cpp 的数值数组与前缀和思路。

7. 删除模式串：P4824 的状态回滚

不能先标出原串所有匹配再统一删除，因为删除后两边会拼接，产生新的匹配。正确做法是边扫描边维护结果栈。

源代码在 Node.j 中保存的是 处理当前字符前、完成失配回退后的 j，所以这个字符被接上后的状态是 Node.j+1。删除模式串后，恢复方式为：

```
j=st.empty() ? 0 : st.top().j+1;
```

更容易记忆的等价方式是直接保存“处理完该字符后的状态”：

```
string ans;
vector<ll> state;
ll j=0,m=pat.size();
for(char c : s){
    while(j!=-1 && c!=pat[j])j=nxt[j];
    j++;
    ans.push_back(c);
    state.push_back(j);
    if(j==m){
        ans.resize(ans.size()-m);
        state.resize(state.size()-m);
        j=state.empty() ? 0 : state.back();
    }
}
```

这个片段依赖非空 pat 及其 nxt。这里不是普通计数匹配：删除后必须取剩余结果的状态，不能直接令 j=nxt[m]。

每个字符至多入栈、出栈一次，时间 O(|s|+|pat|)，空间 O(|s|+|pat|)。当前 Node 的 i,c2 不参与最终匹配和恢复，可以精简。

8. KMP 自动机：把回退预处理掉

go[j][c] 表示已经匹配长度为 j，再读一个字符 c 后的新长度。下面假设字符集为小写英文字母，m=pat.size()，nxt 已计算：

```
vector<vector<ll>> go(m,vector<ll>(26));
for(ll i=0;i<m;i++){
    for(ll c=0;c<26;c++){
        char ch=char('a'+c);
        if(pat[i]==ch){
            go[i][c]=i+1;
        }else if(i==0){
            go[i][c]=0;
        }else {
            go[i][c]=go[nxt[i]][c];
        }
    }
}
```

这里 i>0 时有 0<=nxt[i]<i，依赖的行已经算好，所以预处理是 O(26m)，而不是对每个转移重新跑一遍回退。

只为 0..m-1 建表，但转移结果可以为 m。到达 m 后必须立即按题意处理，不能直接拿 go[m] 继续查。

CF291E：树上继承状态

显式栈保存 {节点编号, 已匹配长度}：

从根的 {1,0} 出发。

每个孩子从父节点的 now 复制出自己的局部 j，逐字符处理边上的字符串。

每次 j==m 就计数，并令 j=nxt[m]，保留重叠匹配。

把处理后的 {v,j} 入栈；兄弟分支之间不能共用修改中的状态。

匹配可以跨过多条边；根到某个后代路径上的一次出现，在其最后字符所在位置计数，不会因为下面还有多个后代而重复计算同一个结束位置。

单一文本的 KMP 可以线性摊还回退；树上同一高状态可能被复制到很多分支，不能直接照搬这份摊还结论。自动机保证每个边字符 O(1) 转移。设所有边字符串总长为 L、节点数为 n，时间 O(26m+L+n)，包含输入存储的空间 O(26m+L+n)。

LeetCode 1397：禁止子串的双边界 DP

状态是 f[i][j][low][high]：

维度	含义
i	已经填入的字符数
j	当前后缀与 evil 前缀匹配的长度

维度	含义
low	当前前缀是否仍与下界 s1 相等
high	当前前缀是否仍与上界 s2 相等

可选字符范围、下一状态与禁止条件：

```
int l=low ? s1[i]-'a' : 0;
int r=high ? s2[i]-'a' : 25;
int nj=go[j][c];
if(nj==m)continue;
int nlow=low && (c==s1[i]-'a');
int nhigh=high && (c==s2[i]-'a');
```

统计出现次数时，匹配完成后回退；禁止出现模式串时，匹配完成就丢弃分支。 回退后继续会把已经出现 evil 的非法字符串算进去。

i==n 返回 1；记忆化数组用 -1 区分未计算与答案为 0。按 1e9+7 取模，时间 O(nm · 26)，DP 空间 O(nm)，另有 O(26m) 自动机和 O(n) 递归栈。

当前源码的 ll f[505][55][2][2] 实际仍在函数内，是约 0.85 MiB 的局部数组；上方“作为成员变量”的注释与代码位置不一致。栈空间较小时应真正移为成员、静态存储或改用动态数组；固定维度也依赖原题范围。

9. 做题时的速查顺序

确定匹配对象。 原串、双倍串、差分数组，还是树上字符流？

确定 j 的含义和 nxt 体系。 已匹配长度；长度式和 pi 式不混用。

确定完整匹配后的动作。 计数回退、返回首个位置、删除并恢复栈状态，或 DP 中丢弃分支。

检查边界。 模式非空、m==1、m>n、重叠匹配、0/1-based 转换、多组状态清空。

检查复用机会。 多询问先全局匹配；多分支或 DP 先建自动机。

KMP 的应用主线可以归纳为：最长 border → 失配回退 → 维护匹配长度 → 按题目改变状态的传递与处理方式。

赛场复用检查

先问自己	检查内容
下標	输入是 0-based 还是 1-based？答案要求哪一种？
范围	`int` 是否足够？求和、乘法、差分是否需要 `long long`？
边界	空串、`m==1`、`m>n`、`n=0/1`、图不连通、重复值是否处理？
状态	多测时容器和全局量是否重置？BFS 入队时是否标记访问？
单调性	二分的 `check` 是否真单调？求最小还是最大可行值？
复杂度	最大输入下时间、空间是否匹配题目限制？
模板改造	源码里的变量范围、字符集、模数、方向和题意是否一致？

训练集中的答案代码是个人经历的证据，不等同于通用模板。遇到题意相同但范围或状态定义不同时，应先改模型再抄代码。